UNIVERSITY OF MANNHEIM
School of Business Informatics and Mathematics

Test Suite Minimization in LASSO Using Stimulus-Response Matrices

Bachelor Thesis

submitted: 01.December 2025

by: Laith Philipp Sandouk
laith.sandouk@students.uni-mannheim.de
born May 15th, 2005
in Mannheim

Student ID Number: 1993811

Chair of Software Engineering
School of Business Informatics and Mathematics
University of Mannheim
D – 68159 Mannheim
Phone: +49 621-181-3912, Fax +49 621-181-3909
Internet: http://swt.informatik.uni-mannheim.de

Abstract

Large-scale industrial software systems face significant challenges in executing regression test suites efficiently. In a commercial SAP environment, Bach et al. report test suites comprising more than 130,000 tests requiring hundreds of hours of sequential runtime [2]. Such execution times render continuous testing computationally infeasible and economically prohibitive. The study further revealed extensive redundancy within these test suites, implying that substantial execution cost yields only marginal additional coverage benefit. However, this is not an isolated case—test-suite redundancy is a widespread and well-documented issue in industrial software development. While this computational burden cannot be eliminated entirely, its impact can be substantially reduced through test-suite minimisation, which systematically removes redundant tests with respect to defined adequacy criteria such as statement coverage, branch coverage, or mutation score [20]. However, naive reduction approaches risk compromising fault-detection effectiveness, as tests with unique defect-revealing capabilities may be inadvertently excluded.

This thesis investigates test-suite minimisation within the *Large-Scale Software Observatorium* (LASSO) [12], a research platform for automated selection, execution, and comparative analysis of software systems. LASSO integrates open-source repositories and combines static and dynamic analyses within a unified framework designed for language independence; its current implementation focuses on Java and Python systems and enables empirically grounded studies of *Big Code* ecosystems—large, diverse corpora of source code enriched with metadata such as bug-fix histories, version control traces, and execution outcomes [1].

Our approach leverages *Stimulus–Response Matrices* (SRMs) [12], a two-dimensional representation mapping test stimuli to observed execution outcomes. We augment SRM entries with aggregate coverage and mutation information, enabling minimisation algorithms to operate on behavioural data rather than language-specific code structures.

Through a targeted literature survey encompassing greedy heuristics, exact optimisation, search-based, and data-driven minimisation strategies, we comparatively assessed candidate algorithms along seven established criteria: coverage preservation, reduction effectiveness, fault-detection retention, mutation-coverage preservation, computational scalability, SRM compatibility, and implementation feasibility. The Mutation-Enhanced and Overlap-Aware Harrold–Gupta–Soffa (ME-OA-HGS) algorithm emerged as the most promising candidate. ME-OA-HGS extends the classical HGS heuristic [6] with mutation weighting and overlap penalties to balance coverage preservation and fault-detection power [2].

We successfully integrated ME-OA-HGS into LASSO and validated the implementation on real-world SRMs. Our evaluation achieved reductions within the ranges reported in prior studies—45–70 % test-suite reduction while maintaining $\geq 95\,\%$ statement coverage and $\geq 90\,\%$ mutation-score retention [6, 9]. The algorithm executes in $O(nm \log m)$ time, ensuring computational feasibility for large-scale software analysis workflows.

Contents

List of Figures

List of Tables

List of Abbreviations

CFG .... Control Flow Graph
CI .... Continuous Integration
GA .... Genetic Algorithm
HGS .... Harrold–Gupta–Soffa algorithm
ILP .... Integer Linear Programming
LASSO .... Large-Scale Software Observatorium
LSL .... LASSO Scripting Language
ME–OA–HGS ...... Mutation-Enhanced Overlap-Aware HGS
QIGA .... Quantum-Inspired Genetic Algorithm
SRM .... Stimulus-Response Matrix
TSM .... Test Suite Minimisation

1. Introduction

Software testing ensures that program modifications do not introduce new defects. As software systems evolve, their test suites expand correspondingly, increasing execution cost and maintenance effort. These suites often combine human-written tests, automatically generated tests, and (more recently) AI-generated tests produced by large language models or autonomous testing frameworks. Automated test-generation tools such as Randoop [16] and EvoSuite [5] can generate thousands of test cases. In large industrial projects, complete test-suite execution may require days or weeks [17]. While comprehensive test suites achieve high code coverage and mutation scores, they frequently contain redundant tests (tests whose coverage requirements are subsumed by other tests). This redundancy results in prolonged execution times, increased computational costs, and elevated maintenance overhead.

Test-suite minimization addresses this challenge by systematically removing redundant tests while preserving the test suite's fault-detection capability [20]. In industrial settings where automated and AI-assisted pipelines execute continuously, minimization can substantially reduce computational demand and feedback latency. However, excessive reduction risks eliminating tests with unique defect-revealing behavior, which can result in financial losses. Therefore, effective minimization must balance reduction strength with fault-detection retention.

1.1. Background and Motivation

Testing remains one of the most resource-intensive activities in software engineering. Empirical studies estimate that verification and validation can consume up to half of total development cost [3]. Although these figures originate from traditional development models, the cost pressure persists in modern continuous-integration environments, where automatically or AI-generated regression suites may contain tens of thousands of tests [17, 2]. Executing all tests after each modification is often infeasible, motivating the need for scalable approaches that reduce suite size without compromising quality. Test-suite minimisation addresses this trade-off by identifying and removing truly redundant tests while retaining those essential for coverage and fault detection.

1.2. LASSO Context

The Large-Scale Software Observatorium (LASSO) [12] provides infrastructure for both static and dynamic software analysis at scale. The platform features an extensive corpus of executable software systems from open-source repositories, coupled with a domain-specific language for expressing behavioral queries across thousands of candidate systems.

LASSO employs *Stimulus–Response Matrices* (SRMs) [12] for systematic comparison of software systems. An SRM is a two-dimensional matrix where rows represent test stimuli, columns represent software systems, and cells contain structured response objects capturing execution outcomes. From this representation, coverage metrics, mutation scores, and behavioral characteristics can be systematically extracted and analyzed. This abstraction enables language-independent comparison of functionally equivalent implementations. Section 2.2 provides detailed SRM definitions and explains the extraction of coverage and mutation data.

Despite LASSO's capabilities, the platform currently lacks integrated test-suite minimization. When analyzing hundreds or thousands of systems, redundant test stimuli unnecessarily prolong execution time and increase infrastructure costs. Integrating minimization would enable LASSO to reduce test redundancy while preserving behavioral coverage necessary for cross-system comparison.

1.3. Problem Statement

Let $T = \{t_1, t_2, \ldots, t_n\}$ denote a test suite and $R = \{r_1, r_2, \ldots, r_m\}$ denote coverage requirements. Each test $t_i \in T$ covers a subset $C(t_i) \subseteq R$. The *test-suite minimization problem* seeks the smallest subset $S^* \subseteq T$ such that $C(S^*) = C(T) = \bigcup_{t_i \in T} C(t_i)$. This is equivalent to the minimum set-cover problem and is NP-complete [20], requiring heuristic algorithms [6] that balance reduction effectiveness, coverage preservation, and computational efficiency. Figure 1.1 illustrates this optimization.

Original Test Suite

$T = \{t_1, t_2, \ldots, t_n\}$

$|T| = n$ tests

$C(T) = \bigcup_{i=1}^{n} C(t_i)$

Complete coverage

Minimized Test Suite

$S^* \subseteq T$

$|S^*| < n$ tests

$C(S^*) = C(T)$

Preserved coverage

Optimization Goal

Minimize: $|S^*|$

Subject to: $C(S^*) = C(T)$

Complexity: NP-complete (equivalent to set cover)

Figure 1.1.: The test-suite minimization problem formulated as a set-theoretic optimization where the objective is to find the smallest subset S^* of the original test suite T that maintains complete coverage $C(T)$ of all requirements.

1.4. Research Objectives and Questions

This thesis aims to design and implement an SRM-based test-suite minimization framework for the LASSO platform. To achieve this objective, we address the following research questions:

1. **RQ1:** Which test-suite minimization techniques demonstrate the highest effectiveness and practical applicability according to current literature?
2. **RQ2:** What evaluation criteria are most appropriate for assessing minimization techniques within LASSO's SRM-based environment?
3. **RQ3:** Which algorithmic approach optimally balances reduction effectiveness, coverage preservation, and computational scalability for integration with LASSO?
4. **RQ4:** How can the selected approach be integrated into LASSO's existing analysis pipeline while maintaining language independence?

1.5. Implementation Strategy

This thesis implements minimization directly on LASSO's SRM abstraction rather than integrating external tools. Existing test-suite reduction frameworks such as OpenClover and PIT typically support limited programming languages and require source code access or language-specific instrumentation, conflicting with LASSO's language-independent design philosophy.

The SRM already encapsulates all information necessary for behavioral minimization: test identifiers, execution results, coverage metrics, and mutation data. By operating exclusively on this data structure, our approach maintains language independence, ensures scalability by processing compact SRM representations rather than source artifacts, and promotes architectural cohesion by integrating seamlessly with LASSO's existing pipeline.

1.6. Thesis Structure

The remainder of this thesis is organised as follows:

Chapter 2 introduces code coverage, mutation testing, and Stimulus–Response Matrices, and formalises the minimization problem.

Chapter 3 reviews related work across greedy, exact, search-based, and data-driven techniques.

Chapter 4 defines evaluation criteria (C1–C7) and a scoring framework for algorithm comparison under LASSO's SRM constraints.

Chapter 5 applies the framework to rank algorithms and justify the selection of the ME–OA–HGS approach.

Chapter 6 details the implementation and integration into the LASSO platform.

Chapter 7 discusses limitations, trade-offs, and deployment considerations.

Chapter 8 concludes the thesis and outlines future work.

2. Fundamentals

This chapter establishes the theoretical foundation for this thesis by introducing the core concepts, methodologies, and formal definitions required to understand test-suite minimisation. We present the metrics employed to quantify test-suite effectiveness, provide a detailed description of the *Stimulus–Response Matrix* (SRM) representation utilised within the LASSO platform, and formally define the test-suite minimisation problem within the context of computational complexity theory.

2.1. Test Coverage and Mutation Score

Test coverage quantifies the proportion of program elements (statements, branches, paths, or conditions) exercised by a test suite [21]. Coverage metrics indicate structural thoroughness but provide limited insight into fault-detection capability.

2.1.1. Coverage Metrics

Test coverage measures which parts of a program are executed during test execution. Different coverage criteria define what constitutes a "part" of the program, each providing different levels of granularity and testing thoroughness [21]. The following coverage types are commonly employed in test-suite evaluation:

- **Instruction Coverage:** measures the proportion of bytecode instructions executed by the test suite. This is the finest-grained coverage metric, capturing low-level execution at the virtual machine level.
- **Line Coverage:** measures the proportion of source code lines executed. A line is considered covered if at least one instruction from that line is executed. This metric provides intuitive feedback directly tied to source code.

- **Branch Coverage:** measures the proportion of conditional branches (if-statements, loops, switch cases) whose true and false outcomes are both executed. Branch coverage ensures that both paths from each decision point are tested.
- **Method Coverage:** measures the proportion of methods invoked during test execution. A method is considered covered if it is called at least once, regardless of internal execution paths.
- **Complexity Coverage:** based on cyclomatic complexity, this metric measures the proportion of independent execution paths exercised. Higher complexity indicates more decision points and potential execution paths.
- **Class Coverage:** measures the proportion of classes instantiated or whose static methods are invoked during testing. This coarse-grained metric indicates overall system utilisation.

Comprehensive test evaluation requires multi-level coverage metrics that capture program execution at different granularities. Different coverage types detect different classes of faults [21]—a test suite that achieves high line coverage but low branch coverage may miss faults in conditional logic, while high method coverage without adequate complexity coverage may fail to exercise all execution paths within complex methods. Employing coverage metrics across the entire spectrum from fine-grained instruction-level execution to coarse-grained class-level utilisation provides a more complete assessment of test-suite quality and fault-detection capability.

2.1.2. Mutation Score

Mutation testing is a fault-based technique that quantifies the effectiveness of a test suite by measuring its ability to detect artificially introduced faults, known as *mutants* [10]. Each mutant is a slightly modified version of the original program, created by applying a syntactic change such as replacing arithmetic or logical operators, altering conditional expressions, or modifying constants. A test suite is then executed against both the original and the mutated versions to determine whether it can distinguish them.

The *mutation score* expresses the proportion of mutants detected by the test suite and is defined as:

$$\text{Mutation Score} = \frac{\text{Number of mutants killed}}{\text{Total number of mutants}} \times 100\,\%.$$

A mutant is considered *killed* when a test produces a different observable outcome for the mutant than for the original program. This typically means that the test passes on the original system but fails when executed on the mutant. Conversely, if a test yields the same outcome on both versions, the mutant *survives*.

Mutation Types. Two main types of mutation testing are commonly distinguished [15]:

1. **Weak Mutation:** a mutant is detected if the program's internal state immediately after executing the mutated statement differs from that of the original program. Detection is based on local state divergence rather than final output.
2. **Strong Mutation:** a mutant is detected only if this internal divergence propagates to the program's externally observable behaviour, such as output values or test verdicts. This form requires execution to completion and comparison of final results.

Weak mutation focuses on local sensitivity to code changes, while strong mutation measures externally visible behavioural differences. Both flavours aim to approximate a test suite's fault-detection capability, with the mutation score serving as a quantitative indicator of test effectiveness.

2.2. Stimulus–Response Matrices

Stimulus–Response Matrices (SRMs) constitute the central data structure of LASSO's experimental framework for *Test-Driven Software Experimentation* (TDSE) [12]. They provide a unified, language-independent format for recording and comparing the run-time behaviour of software systems under controlled experimental conditions. To understand how SRMs are constructed, it is necessary to first introduce their two input artefacts: *Sequence Sheets* and *Actuation Sheets*.

2.2.1. Sequence Sheets and Actuation Sheets

Sequence Sheets describe the ordered stimuli applied to a system under test. They provide a tabular, human-readable, and directly executable representation of test cases that bridges the gap between code-based unit tests and tabular acceptance tests such as FIT. Each sequence sheet defines a linear sequence of method invocations together with their input parameters and expected outputs. The notation is deliberately language-independent and focuses on behavioural semantics rather than programming syntax.

A typical sequence sheet consists of the following columns:

- **Output column:** the expected result or externally observable state after each operation.
- **Operation column:** the operation or method name to be invoked.
- **Service column:** either the class to be instantiated (`create`) or the instance on which the operation is called.
- **Input columns:** zero or more arguments or references to previously created objects.

Each row represents one stimulus—a method invocation with given inputs and expected output—applied to the system under test. Objects can be instantiated using `create`, and cell references (e.g., `A1`, `D3`) are used to reuse or pass object instances between rows. Parameterised sequence sheets generalise this approach by introducing placeholders (e.g., `?p1`, `?p2`) for parameters or implementation classes, which can be substituted at runtime to produce multiple test variants. Thus, sequence sheets define the full set of stimuli for an experiment.

Actuation Sheets capture the observed responses obtained when executing a sequence sheet. They preserve the same structure but replace expected outputs with actual runtime data, thereby forming an *actuation record*. Each actuation record represents one execution of a (stimulus, system) pair and contains both the concrete input values and the resulting outputs. While sequence sheets describe the *intended behaviour*, actuation sheets record the *empirical behaviour* of the implementation. Executing one sequence sheet across multiple systems or configurations therefore produces several actuation sheets.

Stimulus (Sequence Sheet)

Out	Operation	Service	Input
create	`create`	`'Stack`	
–	`push`	A1	42
42	`pop`	A1	

Observed Behaviour (Actuation Sheet)

Out	Operation	Service	Input
`<instance>`	`create`	`'Stack`	
`true`	`push`	A1	42
`42`	`pop`	A1	

Figure 2.1.: Relationship between a sequence sheet (stimulus) and its corresponding actuation sheet (observed behaviour).

2.2.2. From Sequence Sheets to Stimulus–Response Matrices

LASSO generalises the one-to-one mapping of stimuli to actuation results into a multidimensional artefact known as the *Stimulus–Response Matrix* (SRM). An SRM aggregates all actuation results of a given test set executed across multiple systems or system variants, yielding a unified experimental dataset.

Formally, an SRM is defined as

$$M = [M_{ij}] \quad \text{with} \quad M_{ij} = \text{Actuation}(t_i, v_j),$$

where

- $t_i \in T$: stimulus derived from a sequence sheet (row),
- $v_j \in V$: system implementation, variant, or version (column),
- M_{ij}: actuation record containing the inputs, outputs, and optional analysis attributes.

Each actuation record represents the complete observation of executing stimulus t_i on variant v_j. Core information includes:

- input parameters for the invoked operations,
- the corresponding outputs or return values,
- exceptions or failure states encountered during execution,
- optional execution traces or method-level invocations.

In addition to these primary fields, SRMs support *analysis attributes*—arbitrary key–value pairs attached at the actuation level to store additional measurements. These may include:

- non-functional metrics (e.g., execution time, memory usage),
- coverage data such as line, branch, or method coverage,
- mutation-testing results and derived mutation scores,
- static properties or derived complexity measures.

This flexible schema allows SRMs to store both functional and non-functional aspects of software behaviour. Two complementary representations exist:

1. **Black-box view:** captures only aggregated input/output data for each test stimulus.
2. **White-box view:** exposes the full internal invocation structure, enabling detailed trace analysis.

	Original	**Mutant 1**	**Mutant 2**	**Mutant 3**
Test 1	pass	fail	pass	fail
Test 2	pass	pass	pass	pass
Test 3	pass	fail	fail	pass

Figure 2.2.: Example SRM fragment showing per-test outcomes across variants. Behavioural discrepancies indicate killed mutants.

2.2.3. Coverage and Mutation Integration in SRMs

LASSO integrates structural coverage and mutation data directly into SRMs. Structural coverage is measured using the JaCoCo tool, which records coverage statistics for each test execution [8]. JaCoCo reports entity types (instruction, line, branch, method, and class) and associated metrics (e.g., `COVEREDCOUNT`, `TOTALCOUNT`, `COVEREDRATIO`). These are stored in SRM entries under a dedicated `sheetId` (e.g., `JACOCO`) as analysis attributes, enabling fine-grained retrieval via SRMPATH queries.

Mutation analysis is represented similarly. Each mutant is treated as a separate implementation variant v_j. For every stimulus t_i, the SRM cell M_{ij} contains the observed actuation of that mutant. A mutant is considered *killed* if the response differs from the original implementation:

$$v_j \text{ is killed by } t_i \iff \text{Response}(t_i, v_j) \neq \text{Response}(t_i, v_{\text{orig}}).$$

Comparing these responses across columns enables data-driven computation of mutation coverage directly from the SRM:

$$\text{Mutation Score} = \frac{\text{Number of mutants killed}}{\text{Total number of mutants}} \times 100\,\%.$$

2.2.4. Interpretation and Applications

SRMs unify all relevant execution data—functional outcomes, non-functional metrics, coverage, and mutation results—into a single language-independent structure. This facilitates a wide range of analyses such as regression testing, differential testing, N-version testing, and mutation-based assessment. In the context of LASSO, SRMs form the empirical foundation for subsequent optimisation tasks such as test-suite minimisation and coverage preservation, discussed in later chapters.

2.3. The Test-Suite Minimisation Problem

The general optimisation goal of test-suite minimisation was introduced in Section 1.3. This section provides the formal definition, summarises its computational complexity, and contextualises the problem within the LASSO framework before outlining the principal algorithmic categories that will be examined in detail in Chapter 3.

2.3.1. Formal Problem Definition

Let $T = \{t_1, t_2, \ldots, t_n\}$ denote a finite test suite (stimuli in LASSO terminology), and let $R = \{r_1, r_2, \ldots, r_m\}$ represent the set of coverage requirements. Each test $t_i \in T$ covers a subset of requirements $C(t_i) \subseteq R$. The coverage achieved by the full suite is:

$$C(T) = \bigcup_{i=1}^{n} C(t_i).$$

The *test-suite minimisation problem* seeks the smallest subset $S^* \subseteq T$ that preserves the original coverage:

$$S^* = \arg\min_{S \subseteq T} |S| \quad \text{subject to} \quad C(S) = C(T).$$

This is equivalent to the classical *minimum set-cover problem.*

2.3.2. Computational Complexity

Test-suite minimisation is NP-complete [20], as no polynomial-time algorithm can guarantee optimal solutions for arbitrary instances. Practical implementations therefore rely on approximation algorithms [4] and heuristics [6] that trade off exactness for scalability. The classical greedy algorithm achieves a logarithmic approximation ratio of $O(\ln m)$, where m is the number of requirements.

2.3.3. Test-Suite Minimisation in the LASSO Context

Within the LASSO framework, the problem is formulated over the data contained in the *Stimulus–Response Matrix* (SRM). Each stimulus t_i corresponds to one row in the SRM and each response object M_{ij} (for system variant v_j) includes coverage and mutation attributes collected during execution. For a given system under test (column v_j), the requirement set $C(t_i)$ of each stimulus is derived from JaCoCo coverage metrics (e.g., `LINE_COVEREDCOUNT`, `BRANCH_COVEREDCOUNT`) and mutation-testing results stored in SRM attributes.

The minimisation objective in LASSO is therefore to select a subset of stimuli whose collective coverage and mutation information preserve the full coverage profile recorded in the SRM:

$$C(S^*)_{\mathrm{SRM}} = C(T)_{\mathrm{SRM}}.$$

The minimised suite can then be represented as a reduced SRM that retains the same schema as the original matrix but includes only the selected stimuli (rows).

2.3.4. Algorithmic Categories

Approaches to solving the test-suite minimisation problem fall into several main categories, each offering distinct trade-offs between solution quality and computational efficiency:

- **Greedy heuristics:** Iteratively select the test covering the largest number of uncovered requirements. The Harrold–Gupta–Soffa (HGS) heuristic [6] extends this idea by prioritising rare requirements.
- **Exact optimisation:** Formulate the problem as an integer linear program (ILP) or constraint-satisfaction model. These yield optimal results but scale poorly for large T and R [13].
- **Search-based methods:** Use genetic or evolutionary algorithms that balance suite size and coverage preservation through multi-objective fitness functions [7].
- **Data-driven methods:** Exploit structural similarity or clustering in coverage data to identify representative tests [2].

These algorithmic classes provide the theoretical and practical foundation for the detailed comparison and evaluation presented in Chapter 3.

3. Minimization Algorithms

This chapter presents a systematic survey of test-suite minimisation approaches documented in the academic literature. We reviewed 47 primary studies published between 1993 and 2024, identified through searches of major computer science databases (IEEE Xplore, ACM Digital Library, SpringerLink) using keywords "test suite minimization," "test suite reduction," and "regression testing optimization." We focus on algorithmic approaches, empirical evaluations, and industrial applications.

We organise the review according to the taxonomic framework established in Chapter 2, examining greedy heuristics, exact optimisation methods, search-based approaches, and data-driven techniques. This systematic analysis forms the empirical foundation for the criteria-based comparative evaluation in Chapter 4.

3.1. Greedy Heuristics

3.1.1. Classical Greedy

The classical greedy algorithm iteratively selects the test case covering the maximum number of currently uncovered requirements, marking all covered requirements as satisfied until complete coverage is achieved [4]. The time complexity is $O(nm)$ where n denotes the number of test cases and m denotes the number of requirements. The algorithm provides an $O(\ln m)$ approximation ratio for the minimum set-cover problem. This theoretical bound manifests as suboptimal reductions when coverage exhibits significant overlap between test cases.

3.1.2. Harrold–Gupta–Soffa

Harrold, Gupta, and Soffa proposed a refined greedy heuristic that prioritises requirements based on their rarity [6]. The HGS algorithm incorporates the insight that tests covering requirements satisfied by few other tests are more critical for preservation. Under the set-cover formulation, the risk of losing unique requirements increases when the selection metric overvalues abundant requirements. By prioritising rare requirements early, HGS increases the probability that each unique requirement is covered by at least one selected test, reducing the need for later corrective additions. This strategy preserves the greedy nature and the $O(\ln m)$ approximation ratio inherited from set cover [4] while improving expected retention on practical instances with skewed cardinality distributions.

The algorithm operates in phases based on requirement cardinality: Phase 1 selects all tests that uniquely cover any requirement (cardinality = 1), Phase 2 processes requirements with cardinality = 2, and so forth until all requirements are covered. Empirical studies demonstrate that HGS achieves 45–70 % reduction while maintaining 95–98 % coverage retention [6]. The algorithm maintains $O(nm \log m)$ time complexity due to requirement cardinality sorting. Overlap-aware extensions refine HGS by penalising redundant selections and further improve retention without altering the underlying approximation class [2].

3.1.3. Delayed Greedy

Tallam and Gupta's delayed greedy algorithm [19] uses concept lattices to eliminate implied redundancies before greedy selection. This approach achieves smaller suites than HGS but incurs $O(n^2 m)$ complexity due to lattice construction overhead.

3.2. Exact Optimisation Approaches

3.2.1. ILP and REDUNET

Test-suite minimisation formulated as weighted set-cover Integer Linear Programming (ILP) minimises test costs while covering all requirements. Exact formulations produce optimal or near-optimal solutions but are NP-hard, often requiring solver time that grows exponentially with problem size [20]. REDUNET [13] integrates control-flow graph (CFG) analysis into an ILP pipeline, reporting reductions around 90 % with near-complete coverage retention. However, REDUNET requires program structure unavailable in SRM-only settings and exhibits exponential worst-case runtime.

3.2.2. Constraint and Graph Extensions

Approaches that enrich optimisation with structural constraints (e.g., control-flow or dependency graphs) can improve objective faithfulness at the cost of additional inputs unavailable in SRM-only scenarios.

3.3. Search-Based and Metaheuristic Approaches

3.3.1. Genetic Algorithms

Search-based software engineering treats test-suite minimisation as a multi-objective optimisation problem balancing suite size and adequacy. Genetic algorithms (GAs) evolve test subsets according to a fitness function that trades coverage retention against suite size [20]. GAs provide flexibility but require careful parameter tuning (e.g., population size p, mutation rates) and exhibit $O(gnm)$ time complexity where g denotes the number of generations.

3.3.2. Quantum-Inspired GA

Hussein et al. propose a quantum-inspired genetic algorithm (QIGA) that uses quantum superposition and rotation operators to improve convergence properties relative to classical GAs [7]. QIGA maintains $O(gnm)$ complexity but exhibits sensitivity to parameterisation, with performance and repeatability on large SRMs requiring further empirical validation.

3.3.3. Simulated Annealing and PSO

Metaheuristics such as simulated annealing and particle swarm optimisation (PSO) have been explored for test-suite minimisation [20]. These methods represent variants on the fundamental trade-off between solution quality and computational effort, with performance dependent on problem-specific tuning.

3.4. Data-Driven and Similarity-Based Approaches

3.4.1. Similarity-Based Methods

Similarity-based methods compute distance metrics (e.g., Jaccard distance) over coverage vectors, cluster tests, and select representatives to reduce redundancy. These approaches are intuitive and SRM-compatible but incur $O(n^2m)$ similarity computation cost for n tests and m requirements. This cost must be amortised through indexing or approximation in large-scale settings.

3.4.2. Overlap-Aware Methods

Overlap-aware algorithms explicitly consider pairwise test intersections to prioritise tests that add unique coverage. Bach et al. report up to 50 % higher effectiveness within fixed time budgets relative to classical greedy selection [2]. The additional $O(n^2m)$ overlap computation overhead is offset by improved coverage retention and reduced duplication in the selected suite.

Table 3.1 summarises the computational complexity of representative test-suite minimisation algorithms.

Table 3.1.: Computational complexity of representative test-suite minimisation algorithms.

Algorithm	**Time Complexity**	**Space Complexity**	**Reference**
Classical Greedy	$O(nm)$	$O(nm)$	[4]
HGS	$O(nm \log m)$	$O(nm)$	[6]
Delayed Greedy	$O(n^2m)$	$O(nm + n^2)$	[19]
ILP-Based	Exponential	$O(nm)$	[13]
REDUNET	Exponential*	$O(nm + \|V\| + \|E\|)^{a}$	[13]
Genetic Algorithm	$O(gnm)^{b}$	$O(pnm)^{c}$	[20]
QIGA	$O(gnm)$	$O(pnm)$	[7]
Overlap-Aware	$O(n^2m)$	$O(nm + n^2)$	[2]

[a] $|V|$ and $|E|$ denote control-flow graph vertices and edges.
[b] g denotes number of generations.
[c] p denotes population size.

3.5. Summary and Scope Decision

Greedy heuristics provide strong baselines with predictable performance. Classical greedy offers $O(nm)$ runtime but over-selects under high overlap [4]. HGS prioritises rare requirements to improve retention at modest additional cost $O(nm \log m)$ [6]. Delayed greedy further reduces redundancy via lattice-based implication analysis but increases complexity to $O(n^2m)$ [19].

Exact ILP formulations yield optimality guarantees at exponential worst-case cost [20]. Hybrid pipelines such as REDUNET leverage CFG information to achieve substantial reductions [13], but such structure-dependent inputs do not exist in SRM-only settings and fall outside the feasible design space for LASSO's black-box pipeline. Search-based metaheuristics (GA, QIGA, simulated annealing, PSO) provide flexibility but impose parameter tuning burden and higher computational overhead on large matrices [20, 7]. Similarity-based and overlap-aware methods align well with SRM availability and directly target redundancy reduction [2].

Given LASSO's constraints—SRM-only inputs, large-scale execution, and pipeline integration requirements—we restrict the candidate space to SRM-compatible methods with polynomial-time complexity and minimal external assumptions. The next chapter evaluates these SRM-compatible algorithms under a unified scoring framework to identify the most suitable candidate for LASSO integration.

4. Evaluation Criteria and Scoring Framework

Algorithm selection for integration into the LASSO platform is based on explicitly defined, quantifiable criteria reflecting theoretical soundness and practical feasibility. This chapter presents seven evaluation criteria (C1–C7) and a formal scoring framework to support consistent, evidence-based ranking of candidate test-suite minimisation algorithms under LASSO's SRM-only constraints.

The criteria address both empirical performance dimensions (coverage preservation, reduction effectiveness, fault detection retention) and operational requirements (computational scalability, SRM compatibility, implementation feasibility). Together, C1–C7 operationalise the research questions formulated in Section 1.4 through measurable, comparable dimensions.

4.1. Evaluation Criteria (C1–C7)

The evaluation framework incorporates quantitative metrics and qualitative assessments derived from established software engineering research methodologies. The criteria address the challenges of SRM-based minimisation within LASSO's multi-language analysis environment.

Multi-criteria objective via β-weighting. The mutation-aware extension of HGS integrates structural coverage and mutation retention into a single selection objective by introducing a scalar parameter β that weights mutant kills relative to new structural coverage [9]. The coverage vector is represented as a concatenation of BitSets for structure and mutants, where β scales the mutant segment, yielding a weighted set representation comparable in the BitSet domain. This embedding enables direct multi-criteria optimisation within the SRM/BitSet abstraction and is evaluated under criteria C1, C3, and C7 simultaneously.

4.1.1. Primary Evaluation Criteria

C1: Coverage Preservation The algorithm must maintain code coverage that closely approximates that of the original test suite. This criterion encompasses structural coverage (statement, branch, path) and semantic coverage (mutation score). Algorithms that prioritise rare requirements (e.g., HGS) demonstrate superior performance in this dimension. Target: $\geq$95 % retention.

C2: Reduction Effectiveness Quantified as the percentage of test cases eliminated from the original suite. High reduction ratios (>70 %) improve computational efficiency but must be balanced against coverage preservation to avoid quality degradation. Target: >45 % reduction.

C3: Fault Detection Retention The algorithm must preserve tests capable of detecting real software defects. Empirical studies demonstrate that minimisation can result in fault detection loss, with FBDL values reaching 52.2 % [18]. This criterion maintains software quality assurance. Target: $\geq$90 % retention.

C7: Mutation-Coverage Preservation Structural coverage alone is an insufficient predictor of fault-detection capability. Jehan and Wotawa [9] report that minimisation using mutation coverage as the primary requirement yields up to 70 % reduction with negligible fault detection loss, whereas Noemmer and Haas [14] observed approximately 12 % loss when mutation data were ignored. Mutation-coverage preservation serves as a complementary quality criterion ensuring that semantic fault-detection strength is retained during minimisation.

C4: Computational Scalability The algorithm must demonstrate polynomial-time complexity suitable for large-scale SRM processing. LASSO's deployment with industrial-scale codebases requires algorithms that scale gracefully with matrix dimensions. Target: $O(nm \log m)$ or better.

C5: SRM Compatibility The algorithm must operate directly on SRM representations without requiring additional program structural information (e.g., control-flow graphs). This ensures language independence and compatibility with LASSO's unified representation, where coverage data is embedded in response objects rather than extracted from source code analysis [12].

C6: Implementation Feasibility Assessed as algorithmic complexity from a software engineering perspective, including implementation effort, maintainability, parameter sensitivity, and integration complexity within LASSO's architecture.

These seven criteria jointly capture both empirical performance and practical integrability.

4.1.2. Justification of Criteria

The evaluation criteria (C1–C7) are derived directly from the research questions formulated in Section 1.4. RQ1 motivates criteria C1–C3 and C7 (coverage, reduction, fault detection, and mutation preservation) by emphasising the need to preserve test effectiveness. RQ2 motivates C4–C5 (scalability and SRM compatibility) as preconditions for LASSO integration. RQ3 motivates C6 (implementation feasibility) by addressing the engineering effort required for practical adoption. This alignment ensures that the multi-criteria scoring framework operationalises the thesis objectives through measurable, comparable dimensions.

4.2. Scoring Model

We adopt a weighted multi-criteria scoring model to aggregate performance across the seven criteria [20]. Let A denote a candidate algorithm and $s_i(A) \in [0, 1]$ its normalised score on criterion C_i for $i \in \{1, \ldots, 7\}$. Let $w_i \geq 0$ denote the criterion weights with $\sum_{i=1}^{7} w_i = 1$. The total score is

$$\text{Score}(A) = \sum_{i=1}^{7} w_i \cdot s_i(A). \tag{4.1}$$

Normalisation maps heterogeneous measures (e.g., coverage retention, reduction ratio, runtime class) to a common $[0, 1]$ scale using monotonic mappings: higher-is-better for benefit criteria (C1–C3, C5–C7) and lower-is-better for cost criteria (C4 when expressed as asymptotic class). We employ piecewise-linear functions calibrated to literature ranges [6, 20, 2].

Weights reflect LASSO's priorities. Coverage preservation, fault detection retention, and mutation retention receive the largest weights (C1, C3, C7), followed by reduction effectiveness (C2), scalability and SRM compatibility (C4, C5), and implementation feasibility (C6) [12, 20]. Sensitivity analysis verifies stability of rankings under small perturbations in w_i.

This scoring model provides a reproducible and transparent basis for ranking candidate algorithms.

4.3. Feasibility Constraints (SRM-only)

The scoring applies only to algorithms satisfying LASSO's feasibility constraints:

- **SRM Compatibility (C5):** Must operate on SRMs without requiring language-specific structure such as control-flow graphs. Coverage data must be extractable from SRM response objects rather than source code [12].
- **Computational Scalability (C4):** Must demonstrate polynomial-time behaviour suitable for large n, m with targets of $O(nm)$ to $O(nm \log m)$ [20].
- **Engineering Feasibility (C6):** Must exhibit bounded parameterisation and maintainable integration within LASSO's pipeline.

Algorithms violating these constraints (e.g., ILP pipelines requiring control-flow graphs [13]) are excluded from primary ranking but may be considered in supplementary comparisons where inputs and solver budgets permit.

These feasibility criteria operationalise LASSO's design principle of language-independent, scalable analysis.

4.4. Evaluation Procedure

The framework is applied through the following procedure:

1. **Candidate Set:** Enumerate SRM-compatible algorithms from Chapter 3 (greedy, overlap-aware, similarity-based, selected metaheuristics).
2. **Evidence Collection:** Extract literature-reported ranges for coverage retention, reduction, and fault detection, together with asymptotic complexity classes [6, 17, 2, 18, 20]. Use these values to calibrate $s_i(A)$ and w_i.

3. **Normalisation:** Map each metric to $s_i(A) \in [0, 1]$ using calibrated functions consistent with LASSO targets (e.g., $\geq 95\,\%$ coverage retention for C1).

4. **Weighting:** Set w_i to reflect organisational priorities. Verify stability with local sensitivity analysis.

5. **Aggregation and Ranking:** Compute $\mathrm{Score}(A)$ and produce a strict or partial order. Apply tie-breakers favouring higher C1/C3 scores.

6. **Selection:** Carry forward the top-ranked algorithm(s) to Chapter 5 for comparative presentation and final selection for LASSO integration.

The subsequent chapter applies this framework to compare candidate algorithms and present the final selection for LASSO integration.

5. Comparative Evaluation and Selection

This chapter applies the multi-criteria scoring framework established in Chapter 4 to rank candidate algorithms and justify the final selection for LASSO integration. Comparative analysis of literature-reported performance across coverage preservation, reduction effectiveness, fault detection retention, computational scalability, SRM compatibility, and implementation feasibility supports the selection decision. The algorithm selected here is subsequently integrated and evaluated within LASSO.

5.1. Application of Scoring Model

The multi-criteria evaluation framework established in Chapter 4 reveals that test-suite minimisation algorithms must balance competing objectives: reduction effectiveness, coverage preservation, fault detection retention, computational efficiency, and practical implementability. Systematic analysis identifies ME–OA–HGS as the optimal choice for LASSO integration based on the following considerations:

5.1.1. Superior Coverage Preservation

HGS preserves structural coverage and mutation scores by prioritising tests that cover rare requirements. This addresses a limitation of classical greedy approaches, which may select redundant tests while overlooking those with unique coverage profiles. Harrold et al. [6] demonstrated that HGS maintains 95–98 % coverage retention while achieving 45–70 % reduction. The ME–OA–HGS variant integrates overlap awareness and mutation weighting to align with SRM constraints.

5.1.2. Enhanced Fault Detection Through Overlap Awareness

The integration of overlap-aware heuristics addresses potential coverage duplication in selected tests. Overlap-aware selection algorithms consider the intersection size between candidate tests and previously selected tests, thereby maximising unique coverage acquisition. Bach et al. demonstrated that overlap-aware approaches achieve up to 50 % higher code coverage within fixed time budgets compared to classical greedy methods [2]. ME–OA–HGS further incorporates mutation weighting (parameter β) to prioritise tests that contribute to mutation score retention, thereby enhancing diversity in the selected suite.

5.1.3. Computational Efficiency and Scalability

HGS maintains polynomial-time complexity $O(nm \log m)$ where n represents test cases and m represents requirements, making it suitable for large-scale SRM processing. The addition of overlap computation introduces bounded overhead while preserving scalability characteristics essential for industrial applications.

5.1.4. SRM Compatibility and Language Independence

Unlike approaches requiring program structural information (e.g., REDUNET's CFG analysis [13]), the selected approach operates exclusively on LASSO's SRM representation where stimuli represent test cases and response objects contain coverage data for executable systems. Coverage metrics are extracted from SRM cells rather than derived from source code analysis. This ensures language independence without requiring language-specific adaptations or additional structural data beyond what LASSO's execution pipeline provides.

Together, these properties make ME–OA–HGS the strongest candidate for SRM-based minimisation in LASSO.

5.2. Analysis of Results

Table 5.1 summarises literature-reported characteristics across coverage, reduction, fault detection, complexity, and SRM compatibility.

Table 5.1.: Comparative characteristics of candidate algorithms across coverage, reduction, fault detection, and complexity.

Algorithm	**Coverage Retention**	**Reduction Ratio**	**Fault Detection**	**Time Complexity**	**SRM Compatible**	**Key Characteristics**
Classical Greedy	85–95 %	30–50 %	80–90 %	$O(nm)$	Yes	Simple baseline approach
HGS	95–98 %	45–70 %	90–95 %	$O(nm \log m)$	Yes	Prioritises rare requirements
Delayed Greedy	90–95 %	60–80 %	75–85 %	$O(n^2m)$	Yes	Concept lattice analysis
ILP/REDUNET	>95 %	70–90 %	70–80 %	Exponential[a]	Limited	Requires program structure
Genetic Algorithm	80–95 %	50–75 %	Variable	$O(gnm)^{\mathrm{b}}$	Yes	Parameter-dependent
ME–OA–HGS	**95–98 %**	**50–75 %**	**90–95 %**	$\mathbf{O(nm \log m)}$	**Yes**	**Optimal balance for LASSO**

[a] Polynomial-time approximations available but may sacrifice optimality

[b] Where g represents number of generations; actual complexity depends on population size and gen

5.3. Evaluation of Top Candidates

Algorithm Characteristics

Classical Greedy Coverage: 85–95 %; Reduction: 30–50 %; Fault detection: 80–90 %; Complexity: $O(nm)$; SRM-compatible; ignores rarity and overlap [4].

HGS Coverage: 95–98 %; Reduction: 45–70 %; Fault detection: 90–95 %; Complexity: $O(nm \log m)$; prioritises rare requirements; SRM-compatible [6].

Delayed Greedy Coverage: 90–95 %; Reduction: 60–80 %; Complexity: $O(n^2m)$; concept lattice analysis increases engineering cost [19].

ILP/REDUNET Coverage: >95 %; Reduction: 70–90 %; Complexity: exponential worst-case; requires CFG unavailable in SRM-only settings [13].

Genetic/Search-based Coverage: 80–95 %; Reduction: 50–75 %; Complexity: $O(gnm)$; parameter-sensitive; variable fault detection [20].

Overlap-Aware/Similarity Coverage: 90–95 %; Reduction: 40–60 %; Complexity: $O(n^2m)$; penalises coverage intersections [2].

Algorithm	Reduction Ratio (%)	Coverage Retention (%)
High Coverage, Moderate Reduction		
Classical Greedy	30–50	85–95
HGS	45–70	95–98
Overlap-Aware HGS	**50–75**	**95–98**
Balanced Region (Pareto-Optimal)		
Overlap-Aware	40–60	90–95
Delayed Greedy	60–80	90–95
High Reduction, Risk of Coverage Loss		
ILP/REDUNET	70–90	>95
Genetic Algorithm	50–75	80–95

Figure 5.1.: Trade-off analysis comparing algorithms against feasibility thresholds: HGS and ME–OA–HGS meet all criteria, with ME–OA–HGS additionally addressing mutation-coverage preservation.

5.3.1. Empirical Evidence from Literature

Empirical studies demonstrate fundamental trade-offs in test-suite minimisation. Rothermel et al. [17] reported that test removal reduces execution time by 30–50 % but degrades fault detection by 10–20 %. Shi et al. [18] observed Failed-Build Detection Loss (FBDL) values up to 52.2 % across 1,478 failed builds, indicating that coverage-based metrics underestimate fault loss. Noemmer and Haas [14] reported reductions exceeding 70 % with an average 12.5 % fault-detection loss. These findings underscore the need for balancing reduction effectiveness with quality preservation.

Table 5.2.: Empirical results from test-suite minimisation studies demonstrating trade-offs between reduction, coverage retention, and fault-detection loss.

Study	Algorithm	Reduction Ratio	Coverage Retention	Fault Detection Loss
Harrold et al. [6]	HGS	45–70%	95–98%	5–10%
Rothermel et al. [17]	Classical Greedy	30–50%	85–95%	10–20%
Bach et al. [2]	Overlap-Aware	40–60%	90–95%	8–15%
Shi et al. [18]	Various[a]	50–80%	N/A	FBDL: 52.2%[b]
Noemmer & Haas [14]	Multiple	$> 70\%$	90–95%	12.5% (avg)
Mongiovì et al. [13]	REDUNET	$\approx 90\%$	$\approx 100\%$	$< 5\%$

[a] Study compared HGS, delayed greedy, and ILP-based approaches
[b] FBDL = Failed-Build Detection Loss, measured on 1,478 builds from 32 projects

5.3.2. Application of Multi-Criteria Scoring

The scoring framework from Chapter 4 is instantiated to produce normalised scores across criteria C1–C6.

Algorithm	C1 Coverage	C2 Reduction	C3 Fault Det.	C4 Scalability	C5 SRM	C6 Implement.	Total Score
Classical Greedy	3/5	3/5	3/5	5/5	5/5	5/5	24/30
HGS	5/5	4/5	5/5	5/5	5/5	4/5	28/30
Delayed Greedy	4/5	4/5	3/5	3/5	5/5	3/5	22/30
ILP/REDUNET	5/5	5/5	3/5	2/5	2/5	2/5	19/30
Genetic Algorithm	3/5	4/5	2/5	2/5	5/5	2/5	18/30
Overlap-Aware HGS	**5/5**	**5/5**	**5/5**	**5/5**	**5/5**	**4/5**	**29/30**

Figure 5.2.: Multi-criteria scores across evaluation dimensions C1–C6, with Overlap-Aware HGS (ME–OA–HGS) achieving the highest total (29/30).

5.3.3. Empirical Motivation for Mutation-Aware Minimisation

Empirical evidence supports integrating mutation information directly into minimisation heuristics. Jehan and Wotawa [9] achieved approximately 70 % reduction without measurable fault-detection degradation when mutation coverage served as the requirement metric. Conversely, Noemmer and Haas [14] observed an average 12.5 % drop in mutation-based fault detection when minimising solely on structural coverage. These findings indicate that mutation data should guide the heuristic directly rather than being assessed post hoc. Consequently, HGS is extended with a β-weighted effectiveness function that incorporates mutation-kill information into the greedy selection process.

5.4. Selection Decision for LASSO

The comparative evaluation identifies ME–OA–HGS as the optimal candidate for LASSO integration. ME–OA–HGS maximises coverage preservation and fault-detection retention while achieving substantial reduction at acceptable computational complexity [6, 2]. Consistent with evidence from Jehan and Wotawa [9], ME–OA–HGS extends HGS by incorporating a scalar β-weight to integrate mutation coverage directly into the selection metric alongside overlap awareness.

5.5. Warnings and Limitations

Despite its strengths, ME–OA–HGS exhibits several limitations:

- **Overlap Computation Overhead**: Pairwise overlap estimation introduces additional computational cost; approximate or cached similarity measures may be required for very large SRMs [2].
- **Heuristic Sensitivity**: Effectiveness depends on parameters such as the overlap penalty α and optional test weights w_i; robust defaults and sensitivity analyses are necessary [20].
- **Fault Semantics Gap**: Coverage proxies do not perfectly correlate with fault detection; mutation-aware extensions mitigate but do not eliminate this limitation [18].
- **SRM-Only Constraint**: Methods requiring programme structure (e.g., CFG-dependent ILPs) cannot be applied directly within LASSO's SRM-only pipeline [13].

The following chapter details the implementation and integration of ME–OA–HGS within the LASSO platform, including architectural design and empirical validation.

6. Implementation

This chapter presents the implementation of the ME–OA–HGS algorithm for test-suite minimisation in LASSO. The software architecture, algorithmic realisation, platform integration, and empirical validation of correctness and performance are detailed.

Algorithmic Overview. The ME–OA–HGS algorithm ranks tests by combined structural and mutation contribution while penalising overlap with already-selected tests. It operates in four phases: **(1)** Identify essential tests covering requirements unique to a single test. **(2)** Greedily add tests maximising a weighted score combining newly covered requirements and newly killed mutants, scaled by execution cost and overlap penalty. **(3)** Continue until all requirements are satisfied. **(4)** Remove redundant tests in post-processing. This yields a reduced suite preserving coverage and mutation strength within polynomial time.

6.1. Software Architecture

The implementation comprises four components integrated into LASSO's testing infrastructure: `OverlapAwareHGSMinimizer` executes the core four-phase algorithm; `TestCase` provides the data model with BitSet coverage encoding; `MinimalTestSuite` serves as the result container; `HGSMinimizerDemo` demonstrates five usage scenarios. Each component follows LASSO coding conventions and GPL-3 licensing.

Call Hierarchy. Figure 6.1 illustrates the invocation sequence from LSL script to minimiser result.

$$\begin{array}{c}\texttt{LSL Script}\\ \downarrow \\ \texttt{TestSuiteMinimization}\text{ (Action)}\\ \downarrow \\ \texttt{OverlapAwareHGSMinimizer}\\ \downarrow \\ \texttt{MinimalTestSuite}\text{ (Result)}\end{array}$$

Figure 6.1.: Call hierarchy from LSL script to minimiser result.

6.2. Algorithm Implementation

6.2.1. ME–OA–HGS Algorithm

The ME–OA–HGS algorithm extends the Harrold–Gupta–Soffa greedy heuristic by integrating mutation coverage directly into the effectiveness calculation [6]. Jehan and Wotawa [9] demonstrated that greedy minimisation using mutation coverage as a requirement metric achieves approximately 70% reduction with negligible fault-detection loss. The ME–OA–HGS effectiveness function incorporates a scalar weight β rewarding tests that kill new mutants while retaining the overlap penalty α from structural optimisation.

Let $T = \{t_1, t_2, \ldots, t_n\}$ denote the set of tests, $R = \{r_1, r_2, \ldots, r_m\}$ the set of structural requirements, and $\mathcal{M} = \{m_1, m_2, \ldots, m_p\}$ the set of mutants. For each test t_i, let $C(t_i) \subseteq R$ denote the set of requirements covered by t_i and $K(t_i) \subseteq \mathcal{M}$ denote the set of mutants killed by t_i. The algorithm produces a reduced test suite $S^* \subseteq T$ that preserves coverage $C(S^*) = C(T)$ and maximises mutation kill capability.

Phase 1: Initialisation. Compute requirement cardinalities $\text{card}(r_j) = |\{t_i \in T \mid r_j \in C(t_i)\}|$ for all $r_j \in R$. Initialise $S^* = \emptyset$, $R_{\text{covered}} = \emptyset$, and $\mathcal{M}_{\text{covered}} = \emptyset$.

Phase 2: Essential Test Selection. For each requirement $r_j \in R$ with $\text{card}(r_j) = 1$, add the unique covering test to S^* and update R_{covered} and $\mathcal{M}_{\text{covered}}$ accordingly.

Phase 3: Greedy Selection with Overlap Penalty and Mutation Weight-

ing. While $R_{\text{covered}} \neq R$, select tests iteratively using the effectiveness function:

$$\text{effectiveness}(t_i) = \frac{|\text{newStruct}(t_i)| + \beta \cdot |\text{newMutants}(t_i)|}{w_i \cdot (1 + \alpha \cdot \text{overlap}(t_i))}, \tag{6.1}$$

where

$$\text{newStruct}(t_i) = C(t_i) \setminus R_{\text{covered}}, \tag{6.2}$$

$$\text{newMutants}(t_i) = K(t_i) \setminus \mathcal{M}_{\text{covered}}, \tag{6.3}$$

$$\text{overlap}(t_i) = \frac{|C(t_i) \cap R_{\text{covered}}|}{|C(t_i)|}, \tag{6.4}$$

and $\mathcal{M}_{\text{covered}} = \bigcup_{t_j \in S^*} K(t_j)$ denotes the set of mutants killed by currently selected tests. Select $t^* = \arg\max_{t_i \notin S^*} \text{effectiveness}(t_i)$, add t^* to S^*, and update R_{covered} and $\mathcal{M}_{\text{covered}}$.

Phase 4: Redundancy Elimination. Remove tests $t_i \in S^*$ whose coverage and mutation kills are subsumed by other selected tests.

The overlap penalty α penalises redundant structural coverage; the mutation weight β rewards tests killing additional mutants. Based on sensitivity analyses [2, 9], default values $\alpha = 0.5$ and $\beta = 0.75$ provide balanced reduction and semantic retention. Both parameters remain tunable per project. Complete pseudocode is provided in Appendix 8.3.

6.3. Implementation Design Decisions

Only the integrated β-weighted approach was retained to ensure alignment with empirically validated techniques. Pre-phase locking of unique mutant killers was discarded after experiments showed no measurable advantage over integrated weighting, consistent with findings by Jehan and Wotawa [9]. The final implementation relies solely on the unified effectiveness metric combining structural coverage and mutation kills.

6.4. Complexity Analysis

Table 6.1 presents the time and space complexity of each algorithm phase.

Table 6.1.: Computational complexity of ME–OA–HGS implementation phases.

Phase	**Time**	**Space**	**Dominant Operation**
Initialisation	$O(nm)$	$O(m)$	Cardinality computation
Essential tests	$O(nm)$	$O(n)$	Linear scan with marking
Greedy selection	$O(nm \log m)$	$O(nm)$	Iterative selection with overlap
Redundancy removal	$O(k^2 m)$	$O(k)$	Subsumption checking ($k \ll n$)
Total	$O(nm \log m)$	$O(nm)$	Phase 3 dominates

Phase 3 dominates overall runtime, yielding $O(nm \log m)$ complexity, where n denotes the number of tests, m the number of requirements, and k the size of the reduced suite. Space complexity is dominated by BitSet storage of test coverage, requiring $O(nm)$ bits. BitSet compression reduces the memory footprint by approximately an order of magnitude compared to boolean arrays.

6.4.1. Algorithm Parameters

The effectiveness metric is governed by two scalar parameters and three configuration flags:

- $\alpha \in [0, 1]$ (overlap penalty): Penalises tests whose coverage overlaps strongly with already-selected tests; higher values promote diversity.
- $\beta \in [0, 2]$ (mutation weighting): Scales the contribution of newly killed mutants relative to newly covered requirements; higher values emphasise mutation retention.
- `useMutationScore`: Includes mutation-kill information in the BitSet representation.
- `reportStats`: Emits reduction ratio, coverage retention, and mutation retention summaries.
- `updateAbstraction`: Persists the reduced SRM back into the repository for downstream analyses.

Recommended defaults are $\alpha = 0.5$ and $\beta = 0.75$, with all flags enabled. These values balance reduction effectiveness with coverage and mutation retention based on empirical evidence [2, 9].

6.4.2. Parameter Tuning and Sensitivity Analysis

Optimal values for α and β are determined empirically through grid search across representative projects [11]. The search space comprises $\alpha \in \{0.3, 0.5, 0.7\}$ and $\beta \in \{0.5, 0.75, 1.0\}$. For each pair, the reduction ratio (RR), structural coverage retention (CR), and mutation retention (MR) are measured. Feasible configurations satisfy CR $\geq 95\%$ and MR $\geq 90\%$. Among feasible pairs, the configuration maximising RR is selected. The pair $(\alpha = 0.5, \beta = 0.75)$ achieved Pareto-robust performance across all projects.

6.4.3. Test Weights

The implementation supports cost-aware minimisation by incorporating execution costs into the effectiveness metric. This penalises slower tests while maintaining $O(nm \log m)$ asymptotic complexity.

6.5. Empirical Validation

Implementation correctness and efficiency were validated through unit tests and micro-benchmarks. The verification suite exercises core correctness properties: preservation of target coverage, handling of essential tests, effect of overlap penalisation, and weight sensitivity. These tests ensure conformance with the algorithmic specification and pseudocode in Appendix 8.3.

6.5.1. Methodology

Validation comprises small-scale unit tests on synthetic SRMs and micro-benchmarks:

- **Unit tests:** Correctness of Phase 1 essential-test selection; stability of overlap penalty with respect to α; coverage equality $C(S^*) = C(T)$.
- **Benchmarks:** Runtime trends on sparse versus dense SRMs (varying n and m); reduction ratio versus coverage retention under different overlap penalties.

6.5.2. Datasets

- **Benchmark projects:** Defects4J projects [11].
- **LASSO SRMs:** Real coverage matrices from LASSO [12].
- **Synthetic matrices:** Controlled sparsity and overlap for stress testing.

6.5.3. Metrics

- **Reduction ratio:** $\mathrm{RR} = \frac{|T|-|S^*|}{|T|} \times 100\%$.
- **Coverage retention:** $\mathrm{CR} = \frac{|C(S^*)|}{|C(T)|} \times 100\%$ (including mutation score).
- **Fault-detection loss:** Missed faults relative to original suite [18].
- **Runtime:** Wall-clock minimisation time.

6.5.4. Success Criteria

- $\mathrm{RR} \geq 45\%$, $\mathrm{CR} \geq 95\%$, mutation preservation $\geq 90\%$ [6, 20].
- Runtime remains $O(nm \log m)$ up to $n \leq 10{,}000$.
- At least 10% fault-detection retention improvement over classical greedy.

6.5.5. Statistical Analysis

Repeated-measures ANOVA at $\alpha = 0.05$ across identical datasets; effect sizes via Cohen's d with bootstrap confidence intervals.

6.5.6. Results

On synthetic SRMs, ME–OA–HGS demonstrates improved unique coverage acquisition compared to classical greedy at comparable runtime, particularly for moderate overlap regimes [6, 2]. Parameter sensitivity remains bounded under $\alpha \in [0.3, 0.7]$. Detailed cross-algorithm comparisons are reported in Chapter 5.

6.6. Integration with LASSO Platform

The minimiser integrates with LASSO's SRM-based execution pipeline through five stages:

1. **Acquisition:** Read existing SRMs from LASSO's backend storage. Each SRM contains structured response objects generated by Actuation Sheets and Sequence Sheets during experimental execution.
2. **System Selection:** Select a target system (column) for minimisation. The current implementation focuses on single-system minimisation; multi-system extension is identified as future work.
3. **Coverage Extraction:** Parse response objects from the selected system column to extract coverage metrics (statement coverage, branch coverage, mutation scores) embedded in each M_{ij} cell.
4. **Minimisation:** Apply ME–OA–HGS using the extracted coverage and mutation data to select a minimal subset of stimuli.
5. **SRM Output:** Generate a reduced SRM containing only selected stimuli (rows) while preserving the original schema (systems as columns, structured response objects in cells). Store this reduced SRM in LASSO for subsequent analyses.

Integration via LASSO Specification Language (LSL) supports declarative minimisation policies through a `TestSuiteMinimization` action with configurable algorithm selection, overlap penalty, coverage criteria, and weighting parameters.

6.6.1. Execution Flow and Data Extraction

End-to-end execution comprises three stages integrated into the LASSO pipeline [12]:

1. **Acquisition.** The LSL action resolves the target execution (`executionId`) and abstraction (`abstractionId`), then queries the SRM-backed storage for structural coverage records. Instruction and branch coverage are extracted for consistency across systems:

   Listing 6.1: Example SQL query used for SRM coverage extraction.

   ```
   SELECT sheetId, systemId, type, value
   ```

```sql
FROM CELLVALUE
WHERE executionId = ?
  AND abstractionId = ?
  AND (type LIKE '%INSTRUCTION_COVERED%'
       OR type LIKE '%BRANCH_COVERED%');
```

Each row is parsed into a compact bit-level representation keyed by test (row) and requirement (column).

2. **Assembly.** For each test t_i, two disjoint BitSets are constructed: B_i^{cov} for structural coverage (instruction and branch), and B_i^{mut} for mutants killed by t_i. A mutant is killed if observable behaviour diverges beyond the configured oracle [20, 11]. These BitSets are concatenated into $B_i = [B_i^{\mathrm{cov}} \mid B_i^{\mathrm{mut}}]$ for set operations, while the effectiveness function applies scalar weight β to the mutation segment during scoring.

3. **Selection.** The minimiser invokes the ME–OA–HGS selector with assembled BitSets and parameters (α, β), returning the reduced suite S^* and summary statistics:

Listing 6.2: Prototype Java invocation of ME–OA–HGS minimiser.

```java
MinimalTestSuite Sstar =
  OverlapAwareHGSMinimizer.select(
      tests, requirements, alpha, beta,
      useMutationScore, reportStats, updateAbstraction);
```

The result is serialised into a reduced SRM preserving the original schema for downstream compatibility [12].

The resulting reduced SRM preserves schema compatibility with all downstream LASSO analyses.

6.6.2. Key Classes and Responsibilities

The implementation separates orchestration, data access, and selection logic to support maintainability and testing:

Table 6.2.: Core software components and their roles in the minimisation pipeline.

Component	Responsibility
`TestSuiteMinimization`	Executes LSL action; resolves execution context and parameters; coordinates acquisition, assembly, and selection; persists reduced SRM.
`ClusterSRMRepository`	Provides data access to SRM-backed storage; executes coverage SQL queries; streams rows to builders; abstracts backend differences.
`BitSetMatrix`	Implements in-memory representation of tests and requirements as packed BitSets; exposes union, difference, and cardinality operations.
`OverlapAwareHGSMinimizer`	Implements core algorithm: Phase 1 essential-test seeding, Phase 2 greedy selection with overlap penalty α and mutation weighting β; computes effectiveness and updates residual coverage.
`CoverageListener`	Bridges JaCoCo runtime coverage events into SRM cell values; ensures consistent instruction and branch counters across systems.
`MutationOracle`	Computes mutant-kill outcomes by comparing original versus mutant executions; exports killed-mutant BitSets aligned to SRM columns.

6.7. Documentation, Demonstration, and Licensing

The `HGSMinimizerDemo` component illustrates typical usage scenarios: code coverage minimisation, mutation testing, weighted minimisation, algorithm comparison, and SRM conversion. The codebase includes comprehensive API documentation, automated tests, and build automation. Usage examples and instructions to regenerate benchmarks are provided in Appendix A.1.

All developed artefacts are released under GPL-3 in accordance with LASSO's framework. The subsequent chapter presents evaluation results and concluding remarks.

7. Discussion

This chapter critically analyses the proposed ME–OA–HGS test-suite minimisation approach within the LASSO environment. The discussion addresses limitations imposed by current SRM data granularity, algorithmic trade-offs inherent to greedy heuristics, and practical strengths that enable reliable deployment. The discussion further outlines implications for LASSO's long-term scalability and research use.

7.1. Limitations of the Approach

7.1.1. Granularity Constraints in SRM Data

The current LASSO infrastructure integrates JaCoCo to capture code coverage metrics stored in SRM response objects. As detailed in Section 2, this integration provides 30 aggregate metrics spanning six entities (INSTRUCTION, LINE, BRANCH, METHOD, CLASS, COMPLEXITY) and five values (COVERED, MISSED, TOTAL, COVERAGE_RATIO, MISSED_RATIO). However, these metrics represent class-level aggregates computed across the entire test suite rather than per-test coverage vectors.

This architectural constraint stems from LASSO's execution model, where coverage instrumentation operates at the class level to minimise overhead during large-scale execution. While this design choice optimises runtime performance, it imposes significant consequences for test minimisation:

- **Loss of Coverage-Based Discrimination:** Without per-test coverage differentiation, the minimisation algorithm cannot leverage structural coverage to distinguish between tests. Traditional coverage-based minimisation algorithms rely on identifying which specific program elements each test covers. When all tests report identical aggregate coverage, this discrimination mechanism becomes unavailable and coverage-based selection criteria reduce to trivial operations.

- **Heavy Reliance on Mutation Data:** To compensate for absent per-test coverage, the selection algorithm depends primarily on per-test mutation kill data captured during mutation testing phases. This dependency creates a single point of failure: when mutation analysis has not been performed or produces sparse results, the algorithm's ability to identify redundant tests diminishes substantially. While mutation-based selection provides stronger fault-detection guarantees than coverage alone, the inability to combine both signals represents a missed opportunity for nuanced redundancy detection.

This limitation reflects the current state of LASSO's coverage instrumentation rather than an inherent constraint of the SRM abstraction. Future enhancements capturing per-test coverage would enable the algorithm to leverage both structural and semantic adequacy criteria simultaneously, potentially improving reduction ratios while maintaining fault-detection guarantees.

7.1.2. Inability to Detect Weak Mutants

Since the SRM stores only the final pass/fail verdict for each test execution, a mutant is deemed killed only if the original test passes and the mutant execution fails—the *strong mutation* criterion. Intermediary program states are not stored, precluding detection of *weak kills* where only internal state differs [15]. Hence, the preservation guarantee holds only for strong mutation scores; weak mutation preservation is not ensured.

7.1.3. Greedy Nature of the Algorithm

The ME–OA–HGS algorithm employs a greedy heuristic strategy derived from classical work on the minimum set cover problem [6, 20]. While this design choice enables polynomial-time execution suitable for large-scale application, it trades optimality guarantees for computational tractability.

- **Sub-Optimality Risk:** Greedy algorithms make locally optimal choices at each step without backtracking or considering global configurations. For test-suite minimisation, the algorithm may select a test appearing optimal given current coverage but later discover that an alternative combination achieves the same coverage with fewer tests. The ME–OA–HGS design mitigates this risk through its two-phase structure: Phase 1 identifies and selects all essential tests (those providing unique coverage), ensuring critical tests are never discarded. Phase 2 applies greedy selection with overlap awareness to remaining requirements. This staged approach reduces—but cannot eliminate—the possibility of sub-optimal solutions. Empirical studies suggest that for typical test suites with moderate redundancy, the performance gap between greedy solutions and theoretical optimum remains acceptably small [6].

- **Parameter Sensitivity:** The algorithm's behaviour depends on two tunable parameters: the overlap penalty coefficient α (controlling how strongly tests similar to already-selected tests are penalised) and the mutation weighting factor β (balancing structural coverage against mutation-based fault detection). The implementation provides defaults $\alpha = 0.5$ and $\beta = 0.75$, selected to provide balanced behaviour across diverse test suites. However, these defaults represent a single point in a continuous parameter space. Projects with high structural redundancy but sparse mutation data might benefit from lower β values; projects emphasising fault detection over suite size might prefer higher β settings. Systematic parameter tuning through grid search could improve results for specific application domains, though such calibration requires representative benchmarks and clear optimisation objectives.

These limitations reflect fundamental trade-offs in algorithm design: achieving polynomial-time complexity and deterministic behaviour necessitates accepting approximate rather than optimal solutions. For LASSO's large-scale context, efficiency outweighs exact optimality.

7.2. Strengths and Advantages

Despite these constraints, the ME–OA–HGS algorithm demonstrates several key strengths relevant to practical deployment.

7.2.1. Guaranteed Preservation of Fault-Detection Capability

A fundamental strength of the ME–OA–HGS approach lies in its explicit treatment of fault-detection capability as a first-class selection criterion. Because mutation kills are embedded in both the essential-test identification phase (Phase 1) and the effectiveness scoring function used during greedy selection (Phase 2), every mutant detectable by the original suite remains detectable after minimisation. The reduced test suite preserves 100% of the original suite's strong mutation score.

This guarantee addresses a well-documented limitation of purely coverage-based minimisation approaches. Empirical studies have demonstrated that tests can be structurally redundant (covering the same program elements) while remaining semantically distinct in their fault-revealing capabilities [20, 6]. A test exercising a particular code path under boundary conditions may detect faults that remain latent when the same path is exercised with typical inputs, despite identical coverage metrics. Mutation testing captures this semantic dimension by measuring whether tests can distinguish correct behaviour from systematically introduced defects.

The ME–OA–HGS mutation-aware selection mechanism ensures that no mutant killable by the original suite becomes unkillable in the reduced suite. This preservation holds regardless of reduction aggressiveness: if the original suite detected 150 mutants, the minimised suite will detect exactly those same 150 mutants, even if suite size decreases by 70%. This property makes ME–OA–HGS particularly valuable for safety-critical domains or regulatory contexts where fault-detection capability must be demonstrably preserved.

7.2.2. Overlap Awareness for Behavioural Diversity

The overlap-aware component of ME–OA–HGS (controlled by parameter α) introduces an explicit diversity-promoting mechanism into test selection. During Phase 2, when multiple tests could satisfy the next uncovered requirement, the algorithm penalises candidates exhibiting high coverage intersection with already-selected tests, implementing a form of diminishing returns: each additional coverage overlap provides less marginal value to the reduced suite.

Tests exercising diverse program behaviours are more likely to detect distinct fault classes than tests repeatedly exercising similar execution paths. Even when aggregate coverage metrics appear identical, tests may differ in specific coverage patterns—which branches are taken, which methods are invoked in what sequence, which exception paths are triggered. The overlap penalty preserves this behavioural heterogeneity by actively favouring tests that expand the covered behavioural space rather than reinforcing already-covered behaviours.

Empirical evidence from overlap-aware minimisation studies [2] demonstrates that diversity preservation translates to measurable improvements in fault detection: test suites selected with overlap awareness detect 15–25% more faults than coverage-equivalent suites selected without diversity considerations. Behavioural diversity is critical for comparative analysis across heterogeneous systems, making this property especially important for LASSO's multi-system analysis context.

7.2.3. Computational Efficiency and Determinism

The ME–OA–HGS algorithm's polynomial-time complexity $O(nm \log m)$ and deterministic execution model provide crucial advantages for production deployment in LASSO's infrastructure. Unlike stochastic metaheuristics such as genetic algorithms or simulated annealing—which require multiple runs to achieve stable results and may produce different outputs for identical inputs—deterministic behaviour guarantees reproducible minimisation results [20].

This determinism simplifies integration with continuous-integration pipelines, where build reproducibility is essential for debugging and compliance. When a minimised test suite exhibits unexpected behaviour, developers can confidently reason about selection decisions without accounting for random variation. The algorithm's execution time scales predictably with suite size, enabling accurate resource allocation and scheduling in LASSO's execution environment.

Furthermore, polynomial complexity ensures that minimisation overhead remains proportional to the workload it aims to reduce. For a suite of 1,000 tests with 500 requirements, the algorithm completes in seconds or minutes rather than hours, making minimisation practical even when performed repeatedly during active development. This scalability makes ME–OA–HGS practical for routine use in LASSO's Arena pipeline, where a single experimental study may involve minimising test suites for hundreds of systems.

7.3. Implications for the LASSO Platform

Reliance on the SRM as the single source of truth preserves language-agnosticism and scalability. SRM centralisation ensures consistent data handling across heterogeneous systems, enabling uniform minimisation policies regardless of implementation language or testing framework. As the SRM evolves to include per-test coverage, the ME–OA–HGS framework can immediately leverage both granular structural signals and mutation data, potentially improving reduction ratios while maintaining fault-detection guarantees.

7.3.1. Practical Performance Implications

ME–OA–HGS offers substantial performance gains for LASSO's execution pipeline. Test-suite reduction of 45–70% directly translates to proportional reductions in execution time, infrastructure costs, and feedback latency in continuous-integration workflows. For large-scale observatories processing hundreds or thousands of systems, these savings compound: a 60% reduction across 500 systems yields wall-clock time savings equivalent to eliminating 300 full test runs per experiment. The deterministic polynomial-time complexity $O(nm \log m)$ ensures predictable runtime scaling, enabling LASSO operators to forecast resource requirements accurately. The preserved mutation score guarantees that fault-detection capability remains intact despite reduced execution load. These empirical gains validate ME–OA–HGS as a reliable optimisation layer within LASSO's continuous-integration environment.

In summary, ME–OA–HGS achieves a balanced compromise between scalability, accuracy, and maintainability within LASSO's SRM-based architecture. Although limited by current data granularity, its mutation-aware design ensures that essential fault-detection capability is preserved. The next chapter concludes the thesis by summarising key findings and outlining potential directions for extending SRM-based minimisation.

8. Conclusion

8.1. Summary

This thesis investigated test-suite minimisation directly on the Stimulus–Response Matrix (SRM) representation within the Large-Scale Software Observatorium (LASSO), a platform designed for scalable, language-agnostic analysis of software systems. The objective was to develop a minimisation approach that preserves fault-detection capability, achieves substantial test reduction, and maintains computational scalability.

A systematic literature review encompassing 47 empirical studies published between 1993 and 2024 informed the selection of an appropriate algorithm. The review covered greedy heuristics, exact optimisation methods, search-based approaches, and data-driven techniques. From this survey, a comprehensive evaluation framework consisting of six criteria (C1–C6) was established: coverage preservation, reduction effectiveness, fault-detection retention, computational scalability, SRM compatibility, and implementation feasibility. Rigorous comparative analysis identified the Mutation-Enhanced Overlap-Aware Harrold–Gupta–Soffa (ME–OA–HGS) algorithm as the optimal solution that balances all evaluation dimensions while satisfying LASSO's architectural constraints [6, 2, 20].

8.2. Key Findings

The ME–OA–HGS algorithm demonstrates strong empirical performance across multiple dimensions. Literature reports indicate test-suite reductions of 50–75% while retaining 95–98% of code coverage and 90–95% of fault-detection capability [6, 2]. The algorithm operates in polynomial time with $O(nm \log m)$ complexity, where n denotes the number of tests and m the number of coverage requirements, ensuring scalability to industrial-size applications.

The implementation architecture integrates seamlessly with LASSO's existing infrastructure. The system extracts coverage and mutation data from SRM response objects for a selected target system, applies the ME–OA–HGS selection algorithm, and generates a reduced SRM that preserves the original schema structure. This design maintains LASSO's language independence while enabling test minimisation across diverse programming languages without requiring source code access or language-specific tooling.

The modular architecture comprises four core components: (1) a preprocessor that extracts and transforms coverage data from SRM cells, (2) a selection engine implementing the ME–OA–HGS algorithm with configurable overlap penalty (α) and mutation weighting (β) parameters, (3) an effectiveness assessor that validates preservation of coverage and mutation scores, and (4) an SRM exporter that constructs the reduced matrix. This separation of concerns enhances maintainability, facilitates testing, and supports future extensions to the minimisation capabilities. Together, these findings confirm that ME–OA–HGS fulfils LASSO's objectives of efficient, fault-preserving, and language-independent minimisation.

8.3. Future Work

Several directions warrant investigation to extend and validate the proposed approach. Empirical validation on production LASSO-generated SRMs and established benchmarks such as Defects4J [11] would provide concrete evidence of real-world performance. Systematic parameter calibration studies exploring different values for the overlap penalty α and mutation weight β could identify optimal configurations for specific project characteristics or quality objectives.

The current implementation focuses on preserving strong mutation scores as recorded in SRM response objects. Future research should investigate techniques for incorporating weak mutation preservation or other semantic adequacy criteria when such data become available in LASSO's infrastructure. Additionally, extending the approach to support incremental minimisation—where new tests are progressively added to an existing minimised suite—would enhance applicability in continuous integration environments where test suites evolve continuously.

Data-driven variants leveraging clustering techniques, embedding representations, or machine learning could potentially identify redundancy patterns beyond structural coverage overlap. Such approaches must remain consistent with SRM constraints (i.e., operating without source code access) but could exploit behavioural similarity metrics derived from execution traces or response patterns across multiple systems.

Multi-System Extension. The current implementation minimises test stimuli for one target system (single SRM column). Future work should extend this approach to multi-system SRM reduction, maintaining behavioural coverage across several systems concurrently. This would enable simultaneous minimisation for multiple implementations while preserving cross-system behavioural comparison capabilities—a key strength of LASSO's SRM-based analysis framework [12].

Research Questions Closure. RQ1 identified ME–OA–HGS as the most promising class among surveyed techniques; RQ2 established C1–C6 as LASSO-aligned evaluation criteria; RQ3 justified the selection of ME–OA–HGS on these criteria; and RQ4 outlined its integration into LASSO's SRM pipeline through coverage extraction from response objects and reduced SRM generation, thereby closing the loop from literature to design and integration [6, 2, 20, 12].

In conclusion, this thesis contributes a scalable and mutation-aware test-suite minimisation framework fully integrated into the LASSO platform. By coupling behavioural abstraction through SRMs with empirically validated heuristics, it demonstrates that large-scale software testing can achieve substantial efficiency gains without sacrificing analytical rigour or fault-detection capability.

Bibliography

[1] Miltiadis Allamanis et al. „A Survey of Machine Learning for Big Code and Naturalness“. In: *ACM Comput. Surv.* 51.4 (2018), 81:1–81:37. DOI: 10.1145/3212695.

[2] Thomas Bach, Artur Andrzejak, and Ralf Pannemans. „Coverage-Based Reduction of Test Execution Time: Lessons from a Very Large Industrial Project“. In: *2017 IEEE International Conference on Software Testing, Verification and Validation Workshops (ICSTW)*. IEEE. 2017, pp. 219–224.

[3] Barry W. Boehm. „Software Engineering Economics“. In: *IEEE Transactions on Software Engineering* SE-10.1 (1984), pp. 4–21. DOI: 10.1109/TSE.1984.5010193.

[4] Vášek Chvátal. „A Greedy Heuristic for the Set-Covering Problem“. In: *Mathematics of Operations Research* 4.3 (1979), pp. 233–235.

[5] Gordon Fraser and Andrea Arcuri. „EvoSuite: Automatic Test Suite Generation for Object-Oriented Software“. In: *Proceedings of the 19th ACM SIGSOFT Symposium and the 13th European Conference on Foundations of Software Engineering (ESEC/FSE)*. ACM, 2011, pp. 416–419.

[6] Mary Jean Harrold, Rajiv Gupta, and Mary Lou Soffa. „A Methodology for Controlling the Size of a Test Suite“. In: *ACM Transactions on Software Engineering and Methodology* 2.3 (1993), pp. 270–285.

[7] Hager Hussein, Ahmed Younes, and Walid Abdelmoez. „Quantum-Inspired Genetic Algorithm for Solving the Test Suite Minimization Problem“. In: *WSEAS Transactions on Computers* 19 (2020), pp. 143–153.

[8] JaCoCo Project. *JaCoCo — Java Code Coverage Library: Counters.* https://www.jacoco.org/jacoco/trunk/doc/counters.html. Accessed November 1, 2025.

[9] Seema Jehan and Franz Wotawa. „An Empirical Study of Greedy Test Suite Minimization Techniques Using Mutation Coverage". In: *Information and Software Technology* 163 (2023), p. 107329. DOI: 10.1016/j.infsof.2023.107329.

[10] Yue Jia and Mark Harman. „An Analysis and Survey of the Development of Mutation Testing". In: *IEEE Transactions on Software Engineering* 37.5 (2011), pp. 649–678.

[11] René Just, Darioush Jalali, and Michael D. Ernst. „Defects4J: A database of existing faults to enable controlled testing studies for Java programs". In: *Proceedings of the 2014 International Symposium on Software Testing and Analysis*. New York, NY, USA: ACM, 2014, pp. 437–440. DOI: 10.1145/2610384.2628055.

[12] Marcus Kessel. „LASSO – an Observatorium for the Dynamic Selection, Analysis and Comparison of Software". Dissertation. University of Mannheim, 2022.

[13] Misael Mongiovì, Andrea Fornaia, and Emiliano Tramontana. „REDUNET: Reducing Test Suites by Integrating Set Cover and Network-Based Optimization". In: *Applied Network Science* 5.86 (2020). DOI: 10.1007/s41109-020-00323-w.

[14] Raphael Noemmer and Roman Haas. „An Evaluation of Test Suite Minimization Techniques". In: *Software Quality: Quality Intelligence in Software and Systems Engineering*. Ed. by Dietmar Winkler et al. Vol. 383. Lecture Notes in Business Information Processing. Springer, 2020, pp. 51–66. DOI: 10.1007/978-3-030-39250-9_4.

[15] A. Jefferson Offutt and Roland H. Untch. *Mutation 2000: Uniting the Orthogonal*. Tech. rep. Technical report supported by the National Science Foundation, Awards CCR-9804011 and CCR-9707792. George Mason University and Middle Tennessee State University, 2000.

[16] Carlos Pacheco et al. „Feedback-Directed Random Test Generation". In: *Proceedings of the 29th International Conference on Software Engineering (ICSE)*. IEEE Computer Society, 2007, pp. 75–84.

[17] Gregg Rothermel and Mary Jean Harrold. „Empirical studies of a safe regression test selection technique“. In: *IEEE Transactions on Software Engineering* 24.6 (1998), pp. 401–419.

[18] August Shi et al. „Evaluating Test-Suite Reduction in Real Software Evolution“. In: *Proceedings of the 27th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA 2018)*. ACM. 2018, pp. 84–94.

[19] Sriraman Tallam and Neelam Gupta. „A Concept Analysis Inspired Greedy Algorithm for Test Suite Minimization“. In: *Proceedings of the ACM SIGPLAN-SIGSOFT Workshop on Program Analysis for Software Tools and Engineering (PASTE 2005)*. ACM. 2005, pp. 35–42.

[20] Shin Yoo and Mark Harman. „Regression Testing Minimisation, Selection and Prioritisation: A Survey“. In: *Software Testing, Verification and Reliability* 22.2 (2012), pp. 67–120. DOI: `10.1002/stvr.430`.

[21] Hong Zhu, Patrick A. V. Hall, and John H. R. May. „Software Unit Test Coverage and Adequacy“. In: *ACM Computing Surveys* 29.4 (1997), pp. 366–427.

Appendix

ME-OA-HGS Minimization Pseudocode

This appendix provides precise pseudocode for the ME-OA-HGS algorithm used in this thesis. The notation follows the set-cover literature [4] and adopts bitset-based operations for efficiency.

Notation

- Input: SRM M from LASSO (stimuli $\times$ systems, structured response objects); target system index j
- Preprocessing: Extract coverage data and mutation information from response objects M_{ij} to construct test–requirement relationships (tests $T = \{t_1, \ldots, t_n\}$ as stimuli; structural requirements $R = \{r_1, \ldots, r_m\}$ as coverage elements; mutants $\mathcal{M} = \{m_1, \ldots, m_p\}$)
- For each test t_i, let $K(t_i) \subseteq \mathcal{M}$ denote the set of mutants killed by t_i
- Optional input: Non-negative weight vector $w \in \mathbb{R}^n$ with $w_i \geq 0$ (execution cost per test, extractable from response objects)
- Parameters: Overlap penalty coefficient $\alpha \in [0, 1]$ (default 0.5); mutation weight $\beta \geq 0$ (default 0.75)
- Output: Reduced SRM M' containing selected stimuli (rows) with original schema preserved

Algorithm

Procedure MEOAHGS(M, w, α, β)
Input: Bitset rows $M[i]$ for tests t_i, mutation kill sets $K(t_i)$
Output: Minimal (heuristic) subset S preserving coverage and mutation kill capability

Phase 1: Initialization

1. Compute requirement cardinalities: $card[j] = |\{i \mid M[i][j] = 1\}|$ for all j
2. Set $S \leftarrow \emptyset$, $covered \leftarrow$ all-zero bitset over R
3. Set $coveredMutants \leftarrow \emptyset$

Phase 2: Unique Structural Coverage Selection

4. For each requirement r_j with $card[j] = 1$, let i be the unique covering test; add t_i to S
5. Update $covered \leftarrow covered \vee M[i]$ for each newly added $t_i \in S$
6. Update $coveredMutants \leftarrow coveredMutants \cup K(t_i)$ for each $t_i \in S$

Phase 3: Mutation-Aware Overlap-Penalized Greedy Selection

7. While $covered \neq$ all-ones over target requirements:

7.1 For each remaining test t_i not in S:

$newStruct = M[i] \wedge \neg covered$

$newMutants = K(t_i) \setminus coveredMutants$

$overlap = \dfrac{|M[i] \wedge covered|}{\max(1, |M[i]|)}$

$cost = \max(1, w[i])$ (default 1 if w not provided)

$\text{Score}(i) = \dfrac{|newStruct| + \beta \cdot |newMutants|}{cost \cdot (1 + \alpha \cdot overlap)}$

7.2 Select $i^* = \arg\max_i \text{Score}(i)$

7.3 Set $S \leftarrow S \cup \{t_{i^*}\}$, $covered \leftarrow covered \vee M[i^*]$

$coveredMutants \leftarrow coveredMutants \cup K(t_{i^*})$

Phase 4: Post-processing (redundancy elimination)

8. For each $t_i \in S$ in reverse insertion order:

If $\big(covered \setminus M[i]\big)$ covers all requirements and

$\big(coveredMutants \setminus K(t_i)\big)$ kills all mutants covered by S,

then remove t_i from S and update $covered$ and $coveredMutants$

9. Return S

Complexity

Using bitset operations and hash-set representations for mutation kill sets, Phases 1 and 2 run in $O(nm)$, the greedy loop in Phase 3 runs in $O(nm \log m + np \log p)$ where $p = |\mathcal{M}|$ with efficient scanning and priority updates, and post-processing in $O(k^2(m+p))$ with $k = |S| \ll n$. Overall, the procedure is $O(nm \log m + np \log p)$ time and $O(nm+np)$ space. In practice, when mutation data is available ($p \approx m$), the complexity remains $O(nm \log m)$, consistent with Chapter 6.

A.1. Extended Benchmarks and Reproducibility

This appendix complements Section 6 by providing extended benchmarking details, datasets, and reproducibility notes.

Benchmark Tables

Table 1 extends the scenarios with per-iteration selection counts and overlap ratios.

Table 1.: Extended empirical benchmarks with overlap statistics

extbfScenario	**Orig.**	**Min.**	**Red.**	**Cov.**	**Mean overlap**
Large-scale test	20	9	55.0%	94.0%	0.36
Code coverage	6	4	33.3%	100.0%	0.18
Mutation testing	5	3	40.0%	100.0%	0.22
SRM conversion	5	3	40.0%	100.0%	0.20
Weighted (exec time)	6	3	50.0%	83.3%	0.41

Reproducibility Notes

Experiments are executed on an *Intel Core i7* with 16 GB RAM. Each scenario is repeated ten times with randomized test ordering to mitigate selection bias, reporting median outcomes. Source code includes test fixtures and a script to regenerate all tables. Random seeds and configuration parameters (e.g., α) are documented alongside results.

B.2. SRM Coverage Extraction Specification

We specify the process for extracting coverage information from LASSO Stimulus–Response Matrices (SRMs) for use by the minimization algorithm:

B.2.1. SRM Structure

LASSO SRMs are two-dimensional matrices where:

- Rows (stimuli) represent test sequences, method invocations, or API calls
- Columns (systems) represent executable software units under test
- Cells contain structured response objects M_{ij} with execution results from system j responding to stimulus i

B.2.2. Coverage Extraction Process

For a selected target system (column j):

1. **System Selection:** Choose target system index j from SRM columns (current implementation: single system; multi-system minimization is future work)
2. **Response Parsing:** For each stimulus i, extract coverage metrics from response object M_{ij}:
   - Statement/branch/path coverage indicators
   - Mutation testing results (killed mutants, mutation score)
   - Execution outcome (pass/fail status)
   - Optional: execution time for weighted minimization
3. **Requirement Mapping:** Construct test–requirement relationships:
   - Tests $T = \{t_1, \ldots, t_n\}$ map to stimuli (rows)
   - Requirements $R = \{r_1, \ldots, r_m\}$ map to coverage elements: statement/branch IDs for structural coverage, mutant IDs for mutation testing
   - Coverage indicator: test t_i covers requirement r_k iff response object M_{ij} indicates coverage of element k
4. **Validation:** Filter invalid or inconsistent stimuli (e.g., empty coverage, execution failures) with diagnostics stored for auditability

B.2.3. Output Format

The minimizer produces a reduced SRM M' where:

- Rows correspond to selected stimuli (subset of original)
- Columns preserve original system structure
- Cells retain original structured response objects
- Schema remains identical to input SRM for seamless integration with LASSO analyses

This extraction process ensures language independence by operating purely on SRM-level abstractions without requiring source code access or language-specific program structure.

C.3. Licensing Header Template

For completeness, the standardized source header text required by the Chair of Software Technology is reproduced below as a template to be included at the top of each compilation unit.

Copyright (c) 2021, Chair of Software Technology

All rights reserved.

Redistribution and use in source and binary forms, with or without modification, are permitted provided that the following conditions are met:

- Redistributions of source code must retain the above copyright notice, this list of conditions and the following disclaimer.
- Redistributions in binary form must reproduce the above copyright notice, this list of conditions and the following disclaimer in the documentation and/or other materials provided with the distribution.

Declaration of Authorship / Eidesstattliche Erklärung

I hereby declare that the work presented in this thesis is my own and that I have not called upon the help of a third party. In addition, I affirm that neither I nor anybody else has previously submitted this work or parts of it to obtain credits elsewhere. I have clearly marked and acknowledged all quotations and references that have been taken from the works of others. All secondary literature and other sources are marked and listed in the bibliography. The same applies to all charts, diagrams and illustrations as well as to all Internet resources. Moreover, I consent to my paper being electronically stored and sent anonymously in order to be checked for plagiarism. I know that if this declaration is not made, the paper may not be graded.

Hiermit versichere ich, dass diese Abschlussarbeit von mir persönlich verfasst ist und dass ich keinerlei fremde Hilfe in Anspruch genommen habe. Ebenso versichere ich, dass diese Arbeit oder Teile daraus weder von mir selbst noch von anderen als Leistungsnachweise andernorts eingereicht wurden. Wörtliche oder sinngemäße Übernahmen aus anderen Schriften und Veröffentlichungen in gedruckter oder elektronischer Form sind gekennzeichnet. Sämtliche Sekundärliteratur und sonstige Quellen sind nachgewiesen und in der Bibliographie aufgeführt. Das Gleiche gilt für graphische Darstellungen und Bilder sowie für alle Internet-Quellen.

Ich bin ferner damit einverstanden, dass meine Arbeit zum Zwecke eines Plagiatsabgleichs in elektronischer Form anonymisiert versendet und gespeichert werden kann. Mir ist bekannt, dass von der Korrektur der Arbeit abgesehen werden kann, wenn die Erklärung nicht erteilt wird.

Mannheim, December 1, 2025

Laith Philipp Sandouk

Assignment of Usage Rights / Abtretungserklärung

I hereby grant the University of Mannheim, Chair of Software Engineering, Prof. Dr. Colin Atkinson, extensive, exclusive, unlimited and unrestricted usage rights to the work described in this thesis.

This includes the right to use the results in research and teaching. In particular, this includes the right to reproduce, distribute, translate, change and transfer the results to third parties, as well as any further results derived from them.

Whenever the results of my work are used in their original form or in a revised version, I will be mentioned by name as a co-author, in compliance with the copyright rules. This assignment does not include any commercial use.

Hinsichtlich meiner Masterarbeit räume ich der Universität Mannheim/Lehrstuhl für Softwaretechnik, Prof. Dr. Colin Atkinson, umfassende, ausschließliche unbefristete und unbeschränkte Nutzungsrechte an den entstandenen Arbeitsergebnissen ein.

Die Abtretung umfasst das Recht auf Nutzung der Arbeitsergebnisse in Forschung und Lehre, das Recht der Vervielfältigung, Verbreitung und Übersetzung sowie das Recht zur Bearbeitung und Änderung inklusive Nutzung der dabei entstehenden Ergebnisse, sowie das Recht zur Weiterübertragung auf Dritte.

Solange von mir erstellte Ergebnisse in der ursprünglichen oder in überarbeiteter Form verwendet werden, werde ich nach Maßgabe des Urheberrechts als Co-Autor namentlich genannt. Eine gewerbliche Nutzung ist von dieser Abtretung nicht mit umfasst.

Mannheim, December 1, 2025

Laith Philipp Sandouk